PROYECTO INTEGRADOR-PATRONES DE SOFTWARE

Asignatura: Patrones de Software

Sistema de Punto de Venta (POS) integrado con Odoo 20 Community

Especificación académica para estudiantes | Duración: cuatro semanas |

1. Propósito y objetivo

Desarrollar un POS web funcional con frontend completo y una API REST en ASP.NET Core (C#), integrada con Odoo 20 Community. El objetivo central de la asignatura es identificar problemas reales de diseño, investigar alternativas y seleccionar, implementar, probar y justificar patrones de software pertinentes, aplicando Clean Architecture, principios SOLID cuando correspondan y Clean Code.

No se exige una lista de patrones GoF ni un número arbitrario de patrones. Sí se exige evidencia verificable de decisiones de diseño y comprensión de sus consecuencias.

2. Contexto y límites

El POS permite consultar productos, preparar ventas, aplicar descuentos autorizados, confirmar operaciones y consultar resultados. Odoo conserva las funcionalidades empresariales; la solución desarrollada no debe duplicar innecesariamente contabilidad, inventario, facturación electrónica ni procesamiento bancario.

La integración completa con Odoo 20 Community fue estudiada en proyectos de otras asignaturas, pero no se dispone aquí de evidencia de una prueba ejecutada de extremo a extremo para ventas y pagos. Los requisitos que dependan de esas operaciones están condicionados a la validación técnica compartida; no deben presentarse como comprobados.

3. Arquitectura y tecnologías

Componente	Condición
Frontend	Aplicación web completa; React, Vue, Angular, Svelte, Astro, JavaScript Vanilla u otra tecnología justificada. Clean Architecture obligatoria.
Backend	ASP.NET Core Web API en C#, con Clean Architecture obligatoria.
Integración	Frontend → API REST .NET → Odoo 20 Community. El frontend no accede directamente a Odoo.
ERP	Odoo 20 Community instalado localmente, preferentemente con Docker.
Persistencia	Solo los datos propios necesarios; no duplicar registros empresariales sin justificación.
Calidad	Pruebas automatizadas, Git, documentación reproducible y protección de credenciales.

Las responsabilidades Domain, Application, Infrastructure y Presentation deben ser reconocibles en ambas aplicaciones; no se requiere una cantidad fija de carpetas o proyectos. Las dependencias de código deben respetar la dirección hacia las políticas internas.

4. Requisitos funcionales

Código	Requisito
RF-01	Consultar productos disponibles desde Odoo.
RF-02	Buscar productos por nombre o código y mostrar precio, identificación y disponibilidad si existe información fiable.
RF-03	Agregar productos al carrito con cantidades.
RF-04	Modificar cantidades y eliminar productos del carrito.
RF-05	Mostrar subtotales y total coherentes con las reglas de cálculo verificadas.
RF-06	Aplicar políticas de descuento habilitadas.
RF-07	Permitir incorporar o cambiar políticas de descuento sin impacto innecesario sobre otras funcionalidades.
RF-08	Rechazar descuentos no autorizados con mensajes comprensibles.
RF-09	Confirmar una venta con líneas, precios y descuentos.
RF-10	Seleccionar un método de pago configurado.
RF-11	Registrar la venta y el pago en Odoo mediante integración validada.
RF-12	Mostrar identificador, estado y resultado real de la operación.
RF-13	Consultar el historial de ventas registradas.
RF-14	Visualizar el detalle de una venta.
RF-15	Impedir operaciones con datos inválidos o inconsistentes.
RF-16	Evitar registros duplicados por solicitudes repetidas.
RF-17	Gestionar errores de comunicación sin afirmar éxito no confirmado.

Condición técnica: RF-09 a RF-14 dependen especialmente de la comprobación del mecanismo de integración con la instalación Community. Si la interfaz disponible no soporta alguna operación, el docente deberá comunicar el mecanismo validado o ajustar expresamente ese requisito antes de exigirlo.

5. Reglas de negocio mínima

Código	Regla
RN-01	La venta contiene al menos un producto.
RN-02	Las cantidades son mayores que cero.
RN-03	Los precios de referencia se obtienen y verifican con Odoo; el frontend no impone precios arbitrarios.
RN-04	Los descuentos cumplen las políticas autorizadas.
RN-05	No se confirma una venta inconsistente.
RN-06	Una venta confirmada no se edita como venta en preparación.
RN-07	No se informa un pago exitoso sin confirmación del sistema responsable.
RN-08	Una petición repetida no debe generar una segunda venta accidental.

6. Requisitos no funcionales y evidencias

Código	Requisito	Evidencia
RNF-01	Clean Architecture en backend	Revisión de dependencias y responsabilidades.
RNF-02	Clean Architecture en frontend	Casos de uso independientes del framework y HTTP.
RNF-03	Patrones investigados y justificados	Comparación de alternativas, código y pruebas.
RNF-04	SOLID pertinente	Explicación de principios realmente aplicados.
RNF-05	Clean Code	Nombres, cohesión, legibilidad y duplicación razonable.
RNF-06	Pruebas automatizadas	Pruebas unitarias y de integración pertinentes ejecutables.
RNF-07	Extensibilidad	Cambio controlado demostrado.
RNF-08	Errores consistentes	Validaciones y fallos tratados de manera verificable.
RNF-09	Seguridad	Secretos fuera del repositorio y validación de entradas.
RNF-10	Mantenibilidad	Responsabilidades y contratos claros.
RNF-11	Interoperabilidad	Flujo frontend → .NET → Odoo demostrado.

RNF-12	Reproducibilidad	Instrucciones y configuración de ejecución.

7. Investigación y decisiones de diseño

Cada equipo identificará al menos tres decisiones de diseño relevantes del POS. Investigará opciones, incluidos patrones GoF, patrones empresariales y alternativas sencillas sin patrón adicional. Distinguirá estas categorías de Clean Architecture (enfoque arquitectónico), SOLID (principios) y Clean Code (prácticas).

- Para cada decisión: problema concreto; alternativas consideradas; solución elegida y categoría; consecuencias y compromisos; clases o módulos implicados; prueba o demostración verificable.
- No se asignan patrones específicos por anticipado. La cantidad de patrones no incrementa automáticamente la calificación.
- Las decisiones deben surgir de necesidades reales: integración externa, políticas variables, estados, separación de responsabilidades, manejo de errores y testabilidad.
- Cada estudiante debe poder explicar y modificar el código que presenta.

8. Criterios de aceptación

CA-01. El frontend permite ejecutar el flujo completo definido, con navegación, validaciones y mensajes útiles.

CA-02. El backend expone los casos de uso mediante API REST y concentra las reglas pertinentes fuera de los controladores.

CA-03. Las operaciones integradas se verifican contra Odoo; no se confunden datos simulados con registros reales.

CA-04. La estructura y dependencias de Clean Architecture se cumplen en ambas aplicaciones.

CA-05. Se evidencian tres decisiones de diseño investigadas, justificadas y aplicadas.

CA-06. Las reglas RN-01 a RN-08 cuentan con comprobaciones y pruebas pertinentes.

CA-07. Las pruebas se ejecutan mediante instrucciones reproducibles y se reportan sus resultados.

CA-08. La defensa demuestra comprensión individual y respuesta a un cambio de requisito.

9. Plan de cuatro semanas

Semana	Actividades y evidencia
1	Requisitos, investigación inicial de alternativas, diseño arquitectónico, instalación de Odoo y primera consulta integrada.
2	Catálogo, carrito, descuentos, reglas de negocio, frontend y backend; pruebas unitarias iniciales.
3	Ventas, pagos e historial según mecanismo validado; integración, manejo de fallos y pruebas.

4	Refactorización, pruebas finales, documentación, demostración funcional y defensa individual.

10. Entrega final

- Un repositorio Git con frontend, backend, pruebas y archivos de configuración sin secretos.
- README con requisitos previos, instalación, ejecución, variables de entorno, preparación de Odoo y ejecución de pruebas.
- Registro breve de al menos tres decisiones de diseño con alternativas, justificación y evidencias en código.
- Demostración del POS integrado y defensa técnica individual.

11. Rúbrica de evaluación (10 puntos)

Criterio	Puntos
Investigación, selección, implementación y justificación de patrones	
Clean Architecture en frontend y backend	
SOLID, Clean Code y buenas prácticas	
Pruebas automatizadas	
Funcionamiento integral del POS e integración con Odoo	
Defensa técnica individual	
TOTAL	

La evaluación de patrones se basa en el problema resuelto, la adecuación de la solución, su implementación y las evidencias. La defensa puede incluir un cambio pequeño de requisito. Los criterios funcionales sujetos a integración deberán adecuarse al mecanismo efectivamente validado antes de la publicación definitiva.

12. Exclusiones y condición de publicación

No se exige implementar facturación electrónica, pasarelas bancarias reales, contabilidad, inventario completo, múltiples sucursales ni modo sin conexión. Antes de exigir el registro real de ventas y pagos, debe existir un procedimiento de integración validado y reproducible para Odoo 20 Community. La investigación previa se reutiliza y no se repite como nueva auditoría.

Estado documental: especificación académica consolidada; publicación de los requisitos de registro de venta y pago condicionada a la evidencia técnica común de integración.

13. Preparación del entorno e integración

Cada grupo utilizará una instancia local autogestionada de Odoo 20 Community con PostgreSQL y el módulo point_of_sale. Se recomienda Docker. No se requiere modificar el código fuente de Odoo ni desarrollar módulos Python como parte del alcance académico.

- Configurar al menos dos productos vendibles, precios e impuestos aplicables, un punto de venta y un método de pago en efectivo.
- Abrir una sesión POS válida y registrar los identificadores reales de sesión, productos y método de pago.
- Crear un usuario de integración con permisos apropiados; conservar la API key exclusivamente en el backend, fuera del repositorio.
- Comprobar la comunicación HTTP desde .NET hacia Odoo y conservar evidencias de la petición, respuesta y registro observado.
- Realizar durante la primera semana la prueba temprana de registro de una venta con dos productos y pago en efectivo, usando el mecanismo documentado para el POS web.

La prueba de integración se apoya en el análisis técnico ya efectuado para Tecnologías y Desarrollo Web. El método pos.order.sync_from_ui(orders), invocado como candidato mediante JSON-2, debe contrastarse con el código y la configuración de la instancia instalada. Su identificación documental no constituye evidencia de ejecución exitosa. Ante un impedimento reproducible, el grupo notificará al docente con respuestas HTTP y trazas; no podrá sustituir unilateralmente el registro real por un simulador o por el módulo Ventas.

14. Contratos REST de referencia y seguridad

Estas rutas son propuestas para la API .NET; no son endpoints oficiales de Odoo. Cada equipo podrá ajustar sus contratos si justifica la decisión y conserva los casos de uso exigidos.

Método	Ruta propuesta	Propósito
POST	/api/auth/login	Identificación segura del operador
GET	/api/products?search=	Consulta de catálogo
GET	/api/pos/sessions/active	Consulta de sesión abierta
POST	/api/sales	Registro de venta y pago en efectivo
GET	/api/sales	Historial de ventas
GET	/api/sales/{id}	Detalle de una venta

La API .NET deberá comprobar permisos, validar datos de entrada, manejar errores de conexión y respuestas inciertas, y evitar reenvíos que dupliquen ventas. La clave de Odoo nunca llegará al navegador. No se permite acceder directamente desde .NET a PostgreSQL para sustituir operaciones empresariales del POS.

15. Pruebas mínimas y evidencias

Las pruebas unitarias se ejecutarán con dobles de prueba para los contratos de Application. Eso no sustituye la prueba de integración real con Odoo. Los grupos incluirán resultados de ejecución y referencias al código correspondiente.

ID	Nivel	Comprobación mínima
P01	Dominio / Application	Rechazar una venta sin productos.
P02	Dominio / Application	Rechazar cantidad cero o negativa.
P03	Application	Rechazar una sesión POS cerrada o inválida, simulando el contrato externo.

P04	Application	Rechazar un producto no habilitado y evitar el envío de la venta.
P05	Application	No declarar éxito ante error o timeout de Odoo.
P06	Application	Controlar repetición de una operación o devolver resultado incierto sin crear otra venta.
P07	Frontend	Actualizar carrito, cantidades y total provisional.
P08	Frontend	Presentar error comprensible cuando falla la API .NET.
P09	Integración real	Registrar y verificar en Odoo una venta de dos productos y pago en efectivo; se admite procedimiento manual reproducible.
P10	Diseño / extensibilidad	Demostrar con prueba un cambio controlado en una política o mecanismo de integración sin alterar componentes no relacionados.

16. Matriz complementaria de aceptación

Requisito	Comportamiento observable	Evidencia
RF-01–02	Consulta y búsqueda de productos reales; datos comerciales consistentes.	Interfaz y respuesta de Odoo.
RF-03–05	Carrito editable; cantidades válidas; total provisional coherente.	Demostración y P01/P02/P07.
RF-06–08	Descuentos autorizados; restricciones y variación de políticas verificables.	Pruebas y decisión de diseño justificada.
RF-09–12	Venta y efectivo registrados en POS real; identificador, importe y estado verificados.	Respuesta .NET, orden en Odoo y P09.
RF-13–14	Historial y detalle permiten localizar la operación registrada.	Pantallas y consulta API.
RF-15–17	No se aceptan datos inválidos ni éxito falso; se controlan solicitudes repetidas.	P01–P06/P08 y demostración.
RNF-01–05	Separación arquitectónica, decisiones de patrones y código mantenible.	Inspección de dependencias, pruebas y ficha de decisiones.
RNF-06–12	Pruebas ejecutables, seguridad, integración y ejecución reproducible.	Resultados, README, configuración y demostración.

17. Investigación de patrones: evidencia específica de la asignatura

Cada grupo documentará al menos tres decisiones de diseño relevantes. Para cada una indicará: (1) problema concreto, (2) alternativas investigadas, (3) patrón o solución elegida y su categoría, (4) consecuencias y compromisos, (5) ubicación en el código y (6) prueba o demostración. No se exige un patrón GoF específico ni un mínimo artificial de patrones. Se valorará que los estudiantes puedan explicar también cuándo un patrón no aporta valor.

La defensa incluirá un cambio pequeño y controlado de requisitos, elegido por el docente, para comprobar la comprensión de las responsabilidades y la extensibilidad del diseño. Los patrones utilizados en frontend y backend no tienen que ser los mismos.

18. Referencias técnicas y precisiones finales

- Repositorio oficial de Odoo 20, módulo POS:
  https://github.com/odoo/odoo/tree/20.0/addons/point_of_sale
- Código de órdenes POS:
  https://github.com/odoo/odoo/blob/20.0/addons/point_of_sale/models/pos_order.py
- Documentación de API externa JSON-2:
  https://www.odoo.com/documentation/20.0/developer/reference/external_api.html

La evidencia documental no garantiza que la integración funcione en cualquier instalación. Los requisitos de registro real de ventas y pagos se mantienen sujetos al procedimiento técnico común validado. Si resulta necesario cambiar el mecanismo o el alcance, el docente deberá comunicar una decisión uniforme a todos los grupos.

Esta versión conserva el alcance y la rúbrica de Patrones de Software. Los elementos añadidos precisan la preparación técnica, las pruebas y los criterios de aceptación, tomando como referencia el proyecto POS web, sin convertir esta asignatura en una repetición de Tecnologías y Desarrollo Web.